

Movie Recommendation System: Item-Based Collaborative Filtering

Project Objective

In this project, we will build an **item-based collaborative filtering recommender system** using the **Surprise library**. The system will allow users to:

1. Receive movie recommendations based on a specific movie title (e.g., "The Lion King").
2. Understand how item-based collaborative filtering works.
3. Compute item-item similarity scores to recommend movies that are similar in terms of user preferences.

We will:

- Use a real-world movie dataset.
- Preprocess the dataset.
- Train a recommendation model using **KNN-based algorithms**.
- Query the system for recommendations based on movie titles.

The project includes detailed explanations of each step and insights into the underlying concepts.

1. Prerequisites

Libraries Needed

Install the required libraries before starting the project:

- **Surprise**: A library designed specifically for building recommender systems.
 - **Pandas**: For data manipulation and preprocessing.
 - **Numpy**: For numerical computations.
 - **Matplotlib & Seaborn**: For data visualization.
-

2. Dataset Overview

Dataset Requirements

For this project, we will use a movie dataset with the following columns:

1. **userId** : A unique ID representing each user.
2. **movieId** : A unique ID representing each movie.
3. **rating** : The numeric rating (e.g., 1-5 or 1-10) given by a user to a movie.

4. `title` : The name of the movie (used for querying recommendations).

Loading the Dataset

We will use the **MovieLens dataset** (small version), which contains user ratings for movies along with metadata such as movie titles. You can download it from [MovieLens \(https://grouplens.org/datasets/movielens/\)](https://grouplens.org/datasets/movielens/).

Save the dataset locally and load it into your project.

3. Loading and Preparing the Dataset

Step 1: Load the Ratings Data

We first load the dataset containing `userId`, `movieId`, and `rating` columns.

```
In [1]: import pandas as pd

# Load the ratings data
ratings_path = "ratings.csv" # Replace with the path to your ratings file
ratings_df = pd.read_csv(ratings_path)

# Display the first few rows of the dataset
print(ratings_df.head())
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

```
In [2]: ratings_df = ratings_df.drop(columns=['timestamp'], axis=1)
```

```
In [ ]: print(ratings_df.head())
```

	userId	movieId	rating
0	1	1	4.0
1	1	3	4.0
2	1	6	4.0
3	1	47	5.0
4	1	50	5.0

Explanation:

- `pd.read_csv()` loads the dataset into a pandas DataFrame.
- Ensure the dataset contains `userId`, `movieId`, and `rating` columns.

Step 2: Load the Movie Metadata

Next, we load the metadata containing `movieId` and `title`.

In [3]:

```
# Load the movies data
movies_path = "/content/movies.csv" # Replace with the path to your movies f
movies_df = pd.read_csv(movies_path)

# Display the first few rows of the dataset
movies_df.head()
```

Out[3]:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

Step 3: Merge Datasets

We merge the `ratings_df` with `movies_df` to include movie titles in the ratings dataset.

In [4]:

```
# Merge ratings with movie titles
ratings_with_titles = pd.merge(ratings_df, movies_df, on="movieId")

# Display the first few rows of the merged dataset
ratings_with_titles.head()
```

Out[4]:

	userId	movieId	rating	title	genres
0	1	1	4.0	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	1	3	4.0	Grumpier Old Men (1995)	Comedy Romance
2	1	6	4.0	Heat (1995)	Action Crime Thriller
3	1	47	5.0	Seven (a.k.a. Se7en) (1995)	Mystery Thriller
4	1	50	5.0	Usual Suspects, The (1995)	Crime Mystery Thriller

4. Preparing the Data for Surprise

In this step, we need to prepare the movie ratings data to work with the Surprise library. One of the key things the Surprise library needs is the **rating scale** — this simply tells the system what the minimum and maximum possible ratings are in our dataset.

In the dataset, the ratings given by users can range from 0.5 to 5.0 (this is common for movie ratings). **Surprise** needs to know this so it can understand how to process the data correctly.

- **Surprise** needs to know these values to properly interpret the ratings in our dataset and to calculate accurate predictions for movie recommendations.

We use **Surprise's Reader class** to define this rating scale, and then we convert our pandas DataFrame into a Surprise dataset.

`Dataset.load_from_df()` converts the pandas DataFrame into a Surprise-compatible dataset.

In [5]: `!pip install scikit-surprise`

```
Collecting scikit-surprise
  Downloading scikit_surprise-1.1.4.tar.gz (154 kB)
    _____ 154.4/154.4 kB 3.8 MB/s eta 0:0
0:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist
t-packages (from scikit-surprise) (1.4.2)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist
t-packages (from scikit-surprise) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist
t-packages (from scikit-surprise) (1.13.1)
Building wheels for collected packages: scikit-surprise
  Building wheel for scikit-surprise (pyproject.toml) ... done
  Created wheel for scikit-surprise: filename=scikit_surprise-1.1.4-cp311-cp3
11-linux_x86_64.whl size=2505173 sha256=752fb99316be4c7343fe9ae98ffb839c5173c
5eb6e1f144ce403eef51c00cee
  Stored in directory: /root/.cache/pip/wheels/2a/8f/6e/7e2899163e2d85d8266da
ab4aa1cdabec7a6c56f83c015b5af
Successfully built scikit-surprise
Installing collected packages: scikit-surprise
Successfully installed scikit-surprise-1.1.4
```

In []: `!conda install -c conda-forge scikit-surprise`

```
In [6]: from surprise import Dataset, Reader

# Define the rating scale (e.g., 0.5 to 5.0)
reader = Reader(rating_scale=(0.5, 5.0))

# Convert the DataFrame to a Surprise dataset
data = Dataset.load_from_df(ratings_with_titles[["userId", "movieId", "rating"]], reader)
```

```
In [9]: ratings_with_titles['rating'].max()
```

```
Out[9]: 5.0
```

4. Split the Data

In this step, we will **split** the dataset into two parts:

- **Training Set:** 80% of the data will be used to train the recommendation model.
- **Testing Set:** 20% of the data will be used to evaluate the model's performance.

Why do we split the data?

- **Training:** The training set helps the model learn patterns and relationships between users and movies.
- **Testing:** The testing set is used to check how well the model performs when making predictions on new, unseen data. It helps us evaluate the accuracy of the recommendations.

We use **Surprise's** `train_test_split()` function to split the dataset. Here's how we can do that:

```
In [ ]: from surprise.model_selection import train_test_split

# Split the dataset
trainset, testset = train_test_split(data, test_size=0.2, random_state=42)
```

Explanation:

- `train_test_split()` splits the dataset into training and testing sets.
- `test_size=0.2` allocates 20% of the data for testing.

5. Building the Item-Based Collaborative Filtering Model

In this step, we will build the recommendation model using **item-based collaborative filtering**. This approach recommends items (movies) based on the similarities between them.

We will use **KNNBasic**, a popular algorithm from the Surprise library, to compute similarities between items (movies).

Step 1: Configure the Model

We need to configure the model by specifying the **similarity metric** (how items will be compared) and choosing whether the filtering is **user-based** or **item-based**.

Why Item-Based Collaborative Filtering?

- In item-based collaborative filtering, the system recommends items that are similar to the ones the user has already liked. For example, if you liked "The Lion King," the system will recommend movies that other users who liked "The Lion King" also enjoyed.
- **KNNBasic** will compute similarity scores between movies based on how often they are rated similarly by users.

We'll configure the model by choosing a similarity measure (e.g., **cosine similarity**) and setting **user_based=False** to ensure that we are using **item-based** filtering.

```
In [ ]: from surprise import KNNBasic

# Define similarity options
sim_options = {
    'name': 'cosine', # Use cosine similarity to measure the similarity bet
    'user_based': False # Set to False for item-based filtering (True would
}

# Build the model using the KNNBasic algorithm
item_cf_model = KNNBasic(sim_options=sim_options)

# Train the model on the training set
item_cf_model.fit(trainset)
```

Computing the cosine similarity matrix...
Done computing similarity matrix.

Out[13]: <surprise.prediction_algorithms.knns.KNNBasic at 0x2b23a8e6050>

Explanation:

- **sim_options** : Specifies the similarity metric (cosine) and the type of filtering (user_based=False for item-based).
- **KNNBasic** : Computes similarity scores and makes predictions.
- **fit()** : Trains the model on the training set.

Step 2: Model Training Insights

During training, the model computes similarity scores between all pairs of items (movies). This is done by comparing how users rate these items.

For example:

- If many users rate **The Lion King** and **Aladdin** similarly, these movies will have a high similarity score.

What We Can Do:

We can print out the ratings for two movies (e.g., *The Lion King* and *Aladdin*) to see how users rate them, and use that information to understand why the system might compute a high similarity score between them.

```
In [ ]: # Define movie titles
movie_title_1 = "The Lion King"
movie_title_2 = "Aladdin"

# Find the movie IDs for the given titles from the combined DataFrame
movie_id_1 = ratings_with_titles[ratings_with_titles['title'].str.contains(m
movie_id_2 = ratings_with_titles[ratings_with_titles['title'].str.contains(m

# Check if the movie titles were found
if len(movie_id_1) > 0:
    rating_1 = ratings_with_titles[ratings_with_titles['movieId'] == movie_i
    print(f"Movie: {movie_title_1}, Rating: {rating_1}")
else:
    print(f"Movie '{movie_title_1}' not found.")

if len(movie_id_2) > 0:
    rating_2 = ratings_with_titles[ratings_with_titles['movieId'] == movie_i
    print(f"Movie: {movie_title_2}, Rating: {rating_2}")
else:
    print(f"Movie '{movie_title_2}' not found.")
```

```
Movie 'The Lion King' not found.
Movie: Aladdin, Rating: 5.0
```

6. Making Recommendations Based on a Movie Title

In this step, we will define a function that allows us to find movies similar to a given movie title. This will help users understand how item-based collaborative filtering works and how recommendations are generated.

Function to Find Similar Movies

The function will take a **movie title** as input and return a list of the most similar movies based on user ratings. We will use the model trained earlier to find the **most similar items** to the one the user is interested in.

```
In [ ]: def get_similar_movies(movie_title, model, trainset, movies_df, top_n=5):
        # Find the movie ID for the given title
        movie_id = movies_df[movies_df['title'].str.contains(movie_title, case=F

        # Convert the movieId to an internal ID used by Surprise (trainset)
        movie_inner_id = trainset.to_inner_id(movie_id[0])

        # Get the top N most similar movies using the KNN model's get_neighbors
        neighbors = model.get_neighbors(movie_inner_id, k=top_n)

        # Map internal IDs back to movie titles
        similar_titles = [(movies_df[movies_df['movieId'] == int(trainset.to_raw
                           for neighbor in neighbors]

        return similar_titles
```

7. Retrieving Recommendations Based on Movie Title

In the last section, we're calling the `get_similar_movies` function to retrieve recommendations for a given movie title (in this case, "The Lion King"). The process is as follows:

1. Movie Title Input:

We provide the movie title "The Lion King" as an example input.

2. Get Similar Movies:

We use the `get_similar_movies` function to fetch the top 5 similar movies to "The Lion King". This function takes in the movie title, the trained collaborative filtering model (`KNNBasic_model`), the training dataset (`trainset`), and the movie DataFrame (`movies_df`) that holds the movie titles and IDs.

3. Print the Recommendations:

The recommendations are returned as a list of tuples, where each tuple consists of a movie title and its similarity score. If the function successfully returns a list, we print the movie titles along with their similarity scores. If the movie title is not found or there is any issue, the function will return an error message instead.


```
In [ ]: ▾ # Movie title input
movie_title = "Shooter"

# Get the top 5 similar movies
recommended_movies = get_similar_movies(movie_title, item_cf_model, trainset

# Print the recommended movies (only titles)
▾ if isinstance(recommended_movies, list):
    print(f"Top 5 similar movies to '{movie_title}':")
▾     for movie in recommended_movies:
        print(movie)
▾ else:
    print(recommended_movies)
```

Top 5 similar movies to 'Shooter':
Grand Budapest Hotel, The (2014)
Wordplay (2006)
Sahara (1943)
Bend of the River (1952)
Another Year (2010)